

HOW2USE

Python Module Reference Guide

GitRepo · Centile Partners Advisory Ltd

22 July 2026

Overview

This document catalogues all Python programs in the GitRepo across 7 analytical domains. For each module it describes the purpose, how to run it, required inputs, and outputs produced. Scripts are organised by domain; utility/package-init files are omitted.

Domain	# Scripts	Run as
STIR-Engine	12	python / imported
RV-Analysis	9	python / imported
FX Vol Monitoring	2	python
Macro FX Signals Dashboard	14	python / streamlit
Macro Journal	4	streamlit / python
MacroFX Server	6	python / pytest
Utilities & Standalone	4	python / streamlit

Key principle: Research/analysis scripts suit Jupyter notebooks; production pipelines, servers and Streamlit dashboards must remain as .py files.

1. STIR-Engine — Short-Term Interest Rate Trading Engine

End-to-end pipeline for trading SOFR and Fed Funds futures. Combines mean-reversion, momentum and LLM-based OSINT signals with a Markov regime classifier, ensemble blending, and a walk-forward backtester.

File	How to run	Description
stir-engine/main.py	python main.py	Orchestrates the full pipeline: data fetch → signals → regime → backtest. Initialises caches and prints P&L; summary.
stir-engine/config.py	Imported	Central config: API keys (FRED, Anthropic), SOFR futures universe, FRED series list, cache paths. Loads secrets from .env via dotenv.
stir-engine/data/cme.py	Imported	Pulls SOFR/Fed Funds OHLCV via yfinance (CME DataMine fallback). Converts 100-rate convention to implied rate. Returns clean DataFrame.
stir-engine/data/fred.py	Imported	Fetches CPI, unemployment, SOFR fixing from FRED. Requires FRED_API_KEY env var. Functions: fetch_fred_series(), fetch_all_macro().
stir-engine/signals/mean_reversion.py	Imported	OU-process signal: fits half-life, rolling Z-score, emits {-1,0,+1}. Usage: MeanReversionSignal(futures_df, macro_df).fit().compute_signal()
stir-engine/signals/momentum.py	Imported	Price momentum + CFTC COT blend. Fetches weekly COT from CFTC website. Usage: MomentumSignal(futures_df, cot_df).compute_signal()
stir-engine/signals/osint.py	Imported	Claude LLM classifies headlines, scores severity/direction, outputs decay-adjusted Z-score. Modes: 'live' (GDELT/RSS) or 'manual' (CSV backtest). Usage: OsintSignal().load_manual_events('data/events.csv').compute_signal()
stir-engine/regime/classifier.py	Imported	Markov-switching regime classifier (Hamilton 1989) on futures log-returns. 3 regimes: ranging / trending / crisis. Usage: RegimeClassifier(futures_df).fit().smoothed_probs
stir-engine/regime/ensemble.py	Imported	Blends MR + momentum + OSINT using regime-dependent weights. Applies crisis override to suppress technical signals. Usage: Ensemble(combined_df, regime_classifier).compute()
stir-engine/backtest/pnl.py	Imported	P&L; engine: DV01 sizing, transaction costs, slippage (next-day entry), margin. SR3 contract: \$2,500 DV01, \$25/tick. Usage: PnLEngine(ensemble_result, PnLConfig()).compute()
stir-engine/backtest/walk_forward.py	Imported	Walk-forward harness. Refits OU + regime per fold (OOS only). Config: WFCConfig(train=504d, oos=126d, step=63d). Usage: WalkForwardHarness(futures_df, macro_df).run()

2. RV-Analysis — Realised vs Implied Volatility

Compares Yang-Zhang and EWMA realised volatility estimators against implied volatility across FX pairs and tenors. Produces calibration statistics, spread analysis, percentile ranks, and 4-panel PNG dashboards.

File	How to run	Description
rv-analysis/calibrate.py	python calibrate.py --pair EURUSD --days 90 --output-dir results/	Compares YZ and EWMA RV vs 30-min HF benchmark. Outputs RMSE/MAE/correlation and plots.
rv-analysis/test_intraday.py	python test_intraday.py	Fetches today's 30-min EURUSD bars from Bloomberg (Terminal required). Demonstrates blpapi IntradayBarRequest pattern.
rv-analysis/run_usdinr_rsi.py	python run_usdinr_rsi.py --pair USDINR --rsi-period 14 --sell-thresh 75 --cover-thresh 55	RSI mean-reversion backtest on FX pairs. Pulls Bloomberg history, signals short entry at RSI>75, exit at RSI<55. Outputs P&L; chart + stats.
rv-analysis/vol_analysis/main.py	python main.py --pair EURUSD --start 2020-01-01 --end 2024-12-31 --output-dir output/	Full RV vs IV analysis driver. Fetches OHLC + IV from Bloomberg, computes YZ/EWMA for five tenors, spreads, percentile ranks. Saves 4-panel PNGs.
rv-analysis/vol_analysis/config.py	Imported	Static config: tenor map, IV tickers, percentile lookback windows (1Y/2Y), EWMA lambda=0.94, signal thresholds, plot constants.
rv-analysis/vol_analysis/data_fetcher.py	Imported	Bloomberg data layer. Fetches OHLC and implied vol, forward-fills up to 2 days, returns clean DatetimeIndex DataFrames.
rv-analysis/vol_analysis/estimators.py	Imported	RV estimators: Yang-Zhang (3-component) and EWMA (RiskMetrics $\lambda=0.94$). Both return annualised % vol. Usage: yang_zhang_rv(ohlc, window=21).
rv-analysis/vol_analysis/bbg_connector.py	Imported	Bloomberg session singleton + BDH wrapper. bdh(tickers, fields, start, end) is the universal data call. stop_session() for clean teardown.
rv-analysis/vol_analysis/spread_analysis.py	Imported	Computes IV-to-RV spreads and rolling percentile ranks (1Y and 2Y). Returns DataFrame with Spread_YZ, Spread_EWMA, and percentile columns.
rv-analysis/vol_analysis/plotter.py	Imported	Generates 4-panel PNG (vol history / spreads / percentile ranks / snapshot). Usage: plot_dashboard(pair, tenor, data, output_dir).

3. FX Vol Monitoring

Monitors FX implied volatility bands, carry ratios, HAR-RV forecasts and gamma squeeze conditions across G10 and EM pairs over tenors from 1W to 1Y.

File	How to run	Description
fx-vol-bands/fx_vol_bands.py	python fx_vol_bands.py --sample --pair EURUSD python fx_vol_bands.py --blp --pair GBPUSD python fx_vol_bands.py --csv data.csv	Computes 1W implied vol bands, Bollinger width regime filter, gamma squeeze flags. Three data modes: --blp (Bloomberg), --csv FILE, --sample (synthetic GBM). Outputs chart and metrics table.
fx-vol-monitor/fx_vol_carry_monitor.py	python fx_vol_carry_monitor.py --tenor 1M	Multi-tenor vol + carry monitor (1W–1Y). Modules: vol bands (spot ± sigma), HAR-RV forecast, carry/vol ratios, squeeze flags. Bloomberg primary, synthetic fallback. Outputs charts + metrics.

4. Macro FX Signals Dashboard

End-of-day G10 FX signal generation across four pillars: technical, carry, macro, and positioning. Includes a separate implied vs realised vol layer with heatmap dashboards.

File	How to run	Description
macro/main.py	python macro/main.py	EOD orchestrator. Fetches spot, rates, CPI, PMI, positioning from Bloomberg. Builds all four signal pillars, saves to cache, prints summary.
macro/run_jpy_positioning.py	python macro/run_jpy_positioning.py	Standalone JPY positioning script. Pulls 3Y of spot + risk reversals + CFTC. Outputs weekly CSV summary.
macro/config/pairs.py	Imported	Ticker universe: 14 spot pairs, 3M rates, OIS, CPI, PMI, policy rates, risk reversals, CFTC tickers. Defines HIST_DAYS / HIST_YEARS.
macro/config/vol_universe.py	Imported	Implied vol universe: G10+EM pairs, 1W–1Y tenors, spot/IV ticker builders, RV window mapping.
macro/data/bloomberg.py	Imported	Bloomberg data layer: BloombergSession context manager, fetch_spot_history(), fetch_latest_rates(), fetch_macro_history() with periodicity override.
macro/data/cache.py	Imported	Signal caching. save_signals(), load_signals(), get_latest_signal() per pillar.
macro/signals/technical.py	Imported	Momentum (RoC), RSI (14-period), MACD (12/26/9). Outputs {-1,0,+1} per FX pair.
macro/signals/carry.py	Imported	3M rate differential normalised by spot. Outputs carry strength scores.
macro/signals/macro.py	Imported	CPI momentum, PMI above/below 50, rate expectations composite score.
macro/signals/positioning.py	Imported	Risk reversal skew + CFTC leveraged funds % → positioning score (-1 to +1). Includes CFTC percentile rank computation.
macro/vol/data.py	Imported	Vol data layer via xbbg. fetch_implied_history(), fetch_ohlc_history(). Mock/synthetic fallback for development.
macro/vol/signals.py	Imported	Z-score of $\ln(IV/RV_{YZ})$ per pair × tenor. build_snapshot() produces heatmap-ready DataFrame.
macro/vol/estimators.py	Imported	Five RV estimators: Yang-Zhang, close-to-close, Parkinson, Rogers-Satchell, Garman-Klass. All return annualised %.
macro/dashboard/app.py	streamlit run macro/dashboard/app.py	Streamlit dashboard: technical signals heatmap, carry matrix, macro scoreboard. Real-time or mock data mode.
macro/dashboard/vol_app.py	streamlit run macro/dashboard/vol_app.py	Streamlit vol heatmap: IV/RV z-scores per pair × tenor, rich/cheap colour coding, sparklines, drill-down.

5. Macro Journal — Claude-Powered Trade Journal

Streamlit-based workflow for capturing, anonymising and archiving intraday macro market colour. Integrates with the Anthropic Claude API to file raw text, answer questions, and compile end-of-day reports committed to GitHub.

File	How to run	Description
macro_journal/app.py	streamlit run macro_journal/app.py	Main journal UI. Paste raw INFO → Claude files/anonymises → ask questions → close EOD (compiles Markdown report, commits + pushes to GitHub).
macro_journal/claude_client.py	Imported	Anthropic API wrapper. <code>file_info()</code> (anonymise/tag/translate), <code>answer_question()</code> (Q&A; from today's INFO), <code>compile_eod()</code> (compress to Markdown by currency). Loads system prompt from config.
macro_journal/storage.py	Imported	Per-day draft JSON + committed EOD Markdown storage. <code>load_draft()</code> , <code>save_draft()</code> , <code>append_entry()</code> , <code>save_eod_markdown()</code> , <code>list_eod_days()</code> .
macro_journal/git_utils.py	Imported	Silent git integration. <code>commit_and_push(paths, message)</code> . Respects <code>EOD_PUSH</code> env var. Never raises exceptions.

6. MacroFX Server — Backend HTTP Server

Stdlib-only HTTP server backing the MacroFX journal frontend. Handles anonymisation via Claude, currency tagging, per-day JSON storage, EOD report compilation, and Git commit/push. No third-party web framework required.

File	How to run	Description
macrofx/server.py	python macrofx/server.py	Dual-stack HTTP server (port 3170, IPv4+IPv6). Serves static files (public/) and API routes: /api/config, /api/day, /api/file-info, /api/get-question, /api/close-eod.
macrofx/model.py	Imported	Claude API integration. anonymise(text) replaces named banks with regional proxies. compose(text, currency) produces structured prose. Gated on MOCK_MODE env var. Uses claude-opus-4.
macrofx/tagging.py	Imported	Rule-based currency tagger (no model calls). Stages: bracket codes → pairs → aliases → ISO codes. Returns list; first tag is 'primary' for EOD grouping.
macrofx/storage.py	Imported	Per-day JSON file (data/YYYY-MM-DD.json). Atomic writes via temp + os.replace. load_day(), save_day(), append_entry(), close_day(), group_by_primary_tag().
macrofx/git_ops.py	Imported	Git commit + push, never raises. Returns human-readable status string. Respects EOD_PUSH env var.
macrofx/test_tagging.py	python -m pytest macrofx/test_tagging.py	Unit tests for currency tagging: bracket codes, pairs, aliases, standalone ISO codes, evidence filtering, symbol maps (£→GBP, €→EUR, ¥→JPY).

7. Utilities & Standalone Scripts

One-shot analytical scripts and data loaders that operate independently of the main signal pipelines.

File	How to run	Description
macro_charts.py	python macro_charts.py	Plots FRED Real GDP / Total Nonfarm Payrolls (GDP per worker). 2-panel chart: series with trend + CAGR, and YoY % change. Saves PNG to output/macro/gdp_per_worker.png.
margin-debt/fetch_data.py	python margin-debt/fetch_data.py	Downloads FINRA margin statistics Excel, merges FRED GDP + market cap, outputs margin_data.csv. Requires FRED_API_KEY env var.
margin-debt/app.py	streamlit run margin-debt/app.py	Streamlit dashboard: FINRA margin debt normalised to GDP and equity market cap. Vertical event annotations (dot-com peak, GFC, COVID). Requires margin_data.csv (run fetch_data.py first).
liquidity_data.py	python liquidity_data.py --years 2 --source auto	Fetches US liquidity indicators (Fed reserves, ON RRP, TGA, bank credit, dealer net positions, MOVE index) from Bloomberg/FRED. Outputs JSON to dashboards/public/data/liquidity_data.json.

8. Jupyter vs Python — Recommendation

The table below summarises which scripts are best kept as .py files and which would benefit from conversion to Jupyter notebooks.

Keep as .py — cannot or should not be notebooks

File / Category	Reason to keep as .py
macrofx/server.py	HTTP server — must run as a persistent process
macro/dashboard/app.py	Streamlit IS the UI layer
macro/dashboard/vol_app.py	Streamlit IS the UI layer
macro_journal/app.py	Streamlit IS the UI layer
margin-debt/app.py	Streamlit IS the UI layer
All config / data / signal modules	Library code — imported by other scripts
main.py orchestrators	Designed to run headlessly or on a schedule
git_utils / git_ops / storage	Infrastructure/utility code

Convert to Jupyter — analytical/research work

File	Why Jupyter suits it
stir-engine/backtest/pnl.py	Tearsheets, equity curves, iterative tuning
stir-engine/backtest/walk_forward.py	Fold-by-fold diagnostics, cumulative OOS P&L;
stir-engine/signals/mean_reversion.py	OU fit diagnostics, half-life plots, Z-score calibration
rv-analysis/calibrate.py	RMSE/MAE comparison charts inline
rv-analysis/vol_analysis/main.py	Already produces PNGs — natural notebook fit
macro_charts.py	Pure one-shot chart
margin-debt/fetch_data.py + analysis	Research workflow, not a service
liquidity_data.py	Exploratory monitoring and visualisation

Best practice: prototype and iterate in notebooks, then productionise signal logic into importable .py modules that both the notebooks and the production orchestrators can call. This avoids duplication and keeps the Streamlit dashboards and HTTP server intact.